

Problema 1

a) Codificăm o stare specificând poziția agentului și dacă fiecare monedă $\{g_1, g_2, g_3, g_4\}$ a fost colectată sau nu. Putem avea astfel reprezentarea:

$$s = (x_a, y_a, G), \text{ unde}$$

- (x_a, y_a) specifică poziția agentului în labirint;
- $G \subseteq \{g_1, g_2, g_3, g_4\}$ specifică mulțimea de monede rămase de colectat.

Putem folosi și o altă reprezentare de forma $s = (x_a, y_a, b_1, b_2, b_3, b_4)$, unde $b_i = 1$ dacă moneda g_i nu a fost încă colectată și $b_i = 0$ dacă moneda g_i a fost încă colectată.

Starea inițială: $s_0 = (x_{a0}, y_{a0}, \{g_1, g_2, g_3, g_4\})$ sau $s_0 = (x_{a0}, y_{a0}, 1, 1, 1, 1)$.

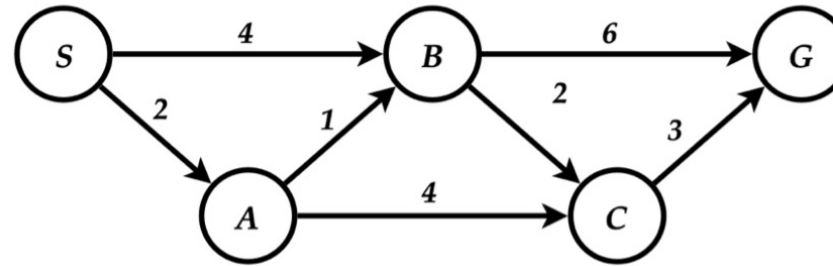
Starea finală: $s_f = (x_{af}, y_{af}, \emptyset)$ sau $s_f = (x_{af}, y_{af}, 0, 0, 0, 0)$.

b) h_1 este admisibilă pentru că agentul trebuie să ajungă măcar la o monedă rămasă, deci costul optim nu poate fi mai mic decât distanța până la cea mai apropiată monedă, deci avem că $h_1(s) \leq h^*(s)$;

h_2 este admisibilă pentru că agentul trebuie să colecteze și cea mai îndepărtată monedă. Fiind bazată pe distanța Manhattan ea subestimează distanța de la agent la monedă într-un labirint cu pereți, deci costul optim nu poate fi mai mic decât distanța până la cea mai îndepărtată monedă, deci avem că $h_2(s) \leq h^*(s)$;

h_3 nu este admisibilă, deoarece poate supraestima costul optim rămas până la colectarea tuturor monedelor. Spre exemplu putem avea monedele toate dispuse într-un colț al labirintului iar în acest caz vom avea $h_3(s) > h^*(s)$.

Problema 2



	h_1	h_2	h_3	h^*
S	8	7	3	8
A	3	4	0	6
B	7	5	4	5
C	2	2	0	3
G	0	0	0	0

a. S

S-A, S-B

S-A-B, S-A-C, S-B

S-A-B, S-A-C, S-B-C, S-B-G

Soluție căutare în lățime: S-B-G

(în implementarea cea mai eficientă verificăm dacă am ajuns la un nod scop când generăm succesorii)

b. Putem să adăugăm **coloana cu valorile exacte $h^*(s)$ pentru fiecare stare s**

Se observă din tabel, comparând coloanele h_1 , h_2 , h_3 cu coloana h^* că h_2 și h_3 sunt admisibile dar h_1 nu este admisibilă întrucât $h_1(B) = 7 > h^*(B) = 5$.

c.

S(3)

S-A(2+0=2), S-B(4+4=8)

S-A-C(6+0=6), S-A-B(3+4=7), S-B(8)

S-A-B(7), S-B(8), S-A-C-G(9+0=9)

S-A-B-C(5+0=5), S-B(8), S-A-B-G(9+0=9), S-A-C-G(9)

S-B(8), S-A-B-C-G(8+0=8), S-A-B-G(9), S-A-C-G(9)

S-B-C(6+0=6), S-A-B-C-G(8), S-A-B-G(9), S-A-C-G(9), S-B-G(10)

S-A-B-C-G(8), S-A-B-G(9), S-A-C-G(9), S-B-C-G(9), S-B-G(10)

d.

S(8)

S-A(2+3=5), S-B(4+7=11)

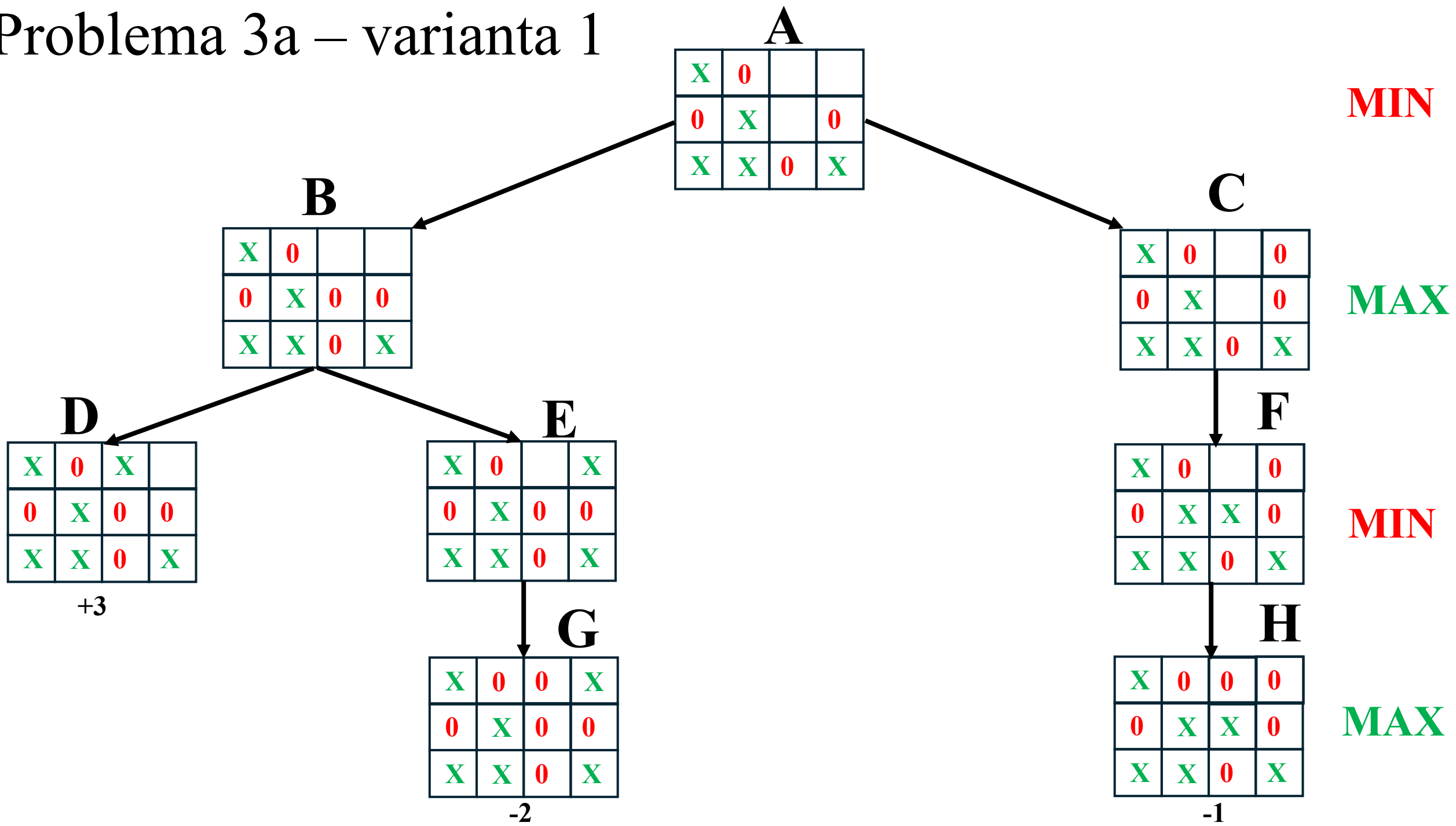
S-A-C(6+2=8), S-A-B(3+7=10), S-B(11)

S-A-C-G(2+4+3=9), S-A-B(10), S-B(11)

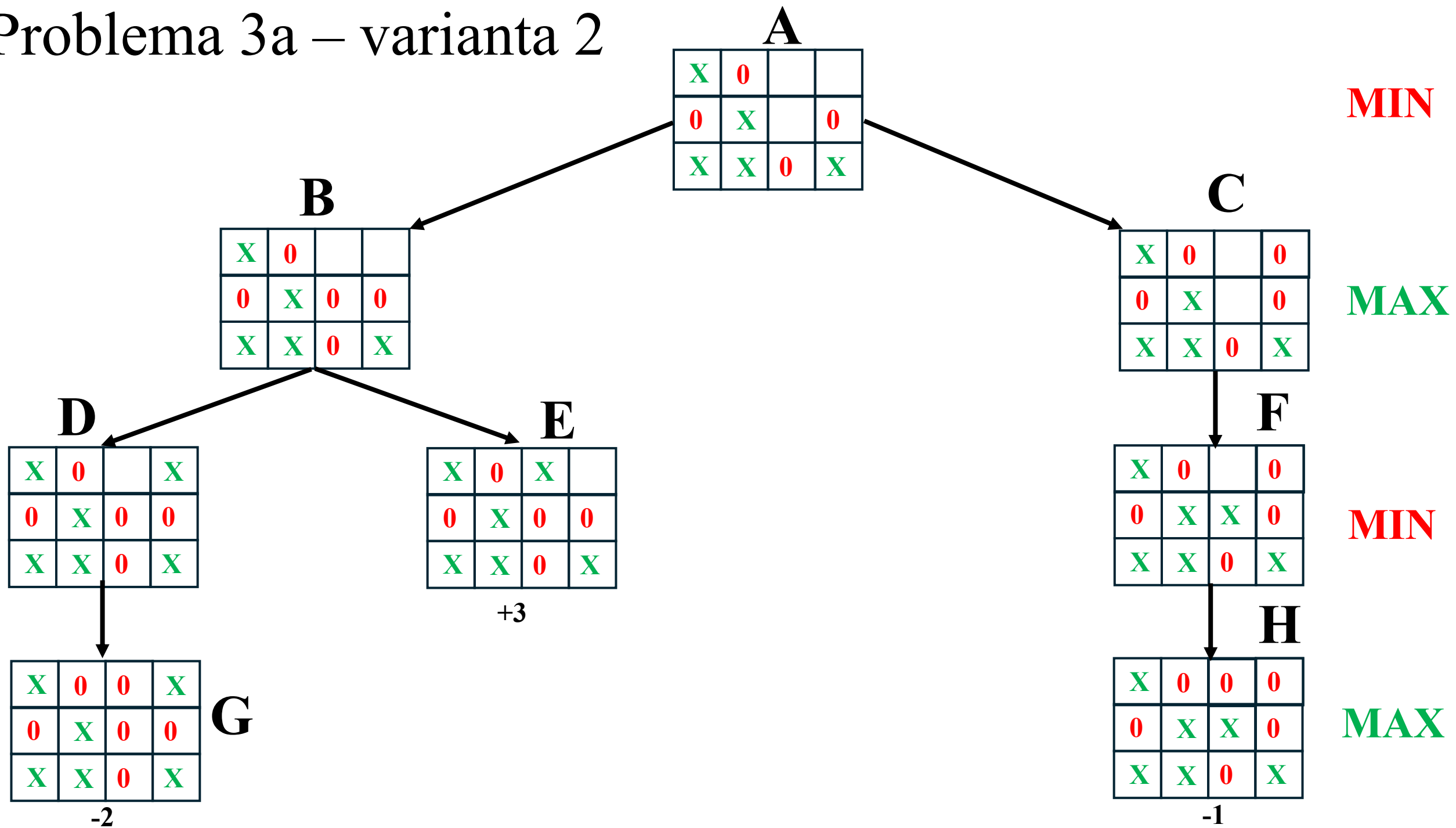
Solutia S-A-C-G, solutie neoptimă, se poate întâmpla întrucât h_1 nu e admisibilă

Solutia S-A-B-C-G, solutie optimă garantată pentru că h_3 e admisibilă

Problema 3a – varianta 1



Problema 3a – varianta 2



Problema 3a – varianta 3

MIN

A

X	0		
0	X		0
X	X	0	X

B

X	0		0
0	X		0
X	X	0	X

C

X	0		
0	X	0	0
X	X	0	X

MAX

D

X	0		0
0	X	X	0
X	X	0	X

E

X	0		X
0	X	0	0
X	X	0	X

F

X	0	X	
0	X	0	0
X	X	0	X

MIN

+3

MAX

G

X	0	0	0
0	X	X	0
X	X	0	X

-1

H

X	0	0	X
0	X	0	0
X	X	0	X

-2

Problema 3a – varianta 4

MIN

A

X	0		
0	X		0
X	X	0	X

B

X	0		0
0	X		0
X	X	0	X

C

X	0		
0	X	0	0
X	X	0	X

MAX

D

X	0		0
0	X	X	0
X	X	0	X

E

X	0	X	
0	X	0	0
X	X	0	X

F

X	0		X
0	X	0	0
X	X	0	X

MIN

+3

H

X	0	0	X
0	X	0	0
X	X	0	X

MAX

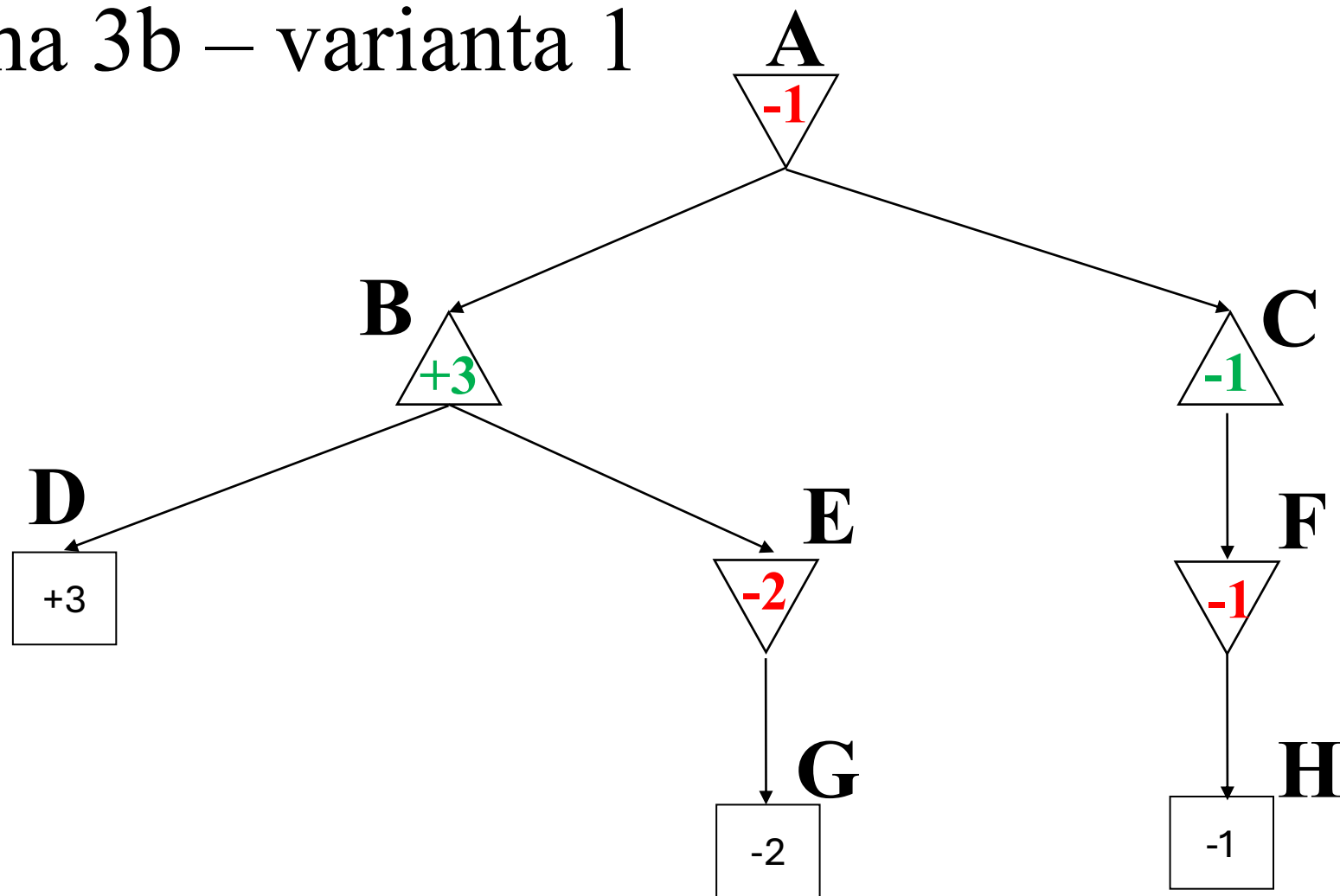
G

X	0	0	0
0	X	X	0
X	X	0	X

-1

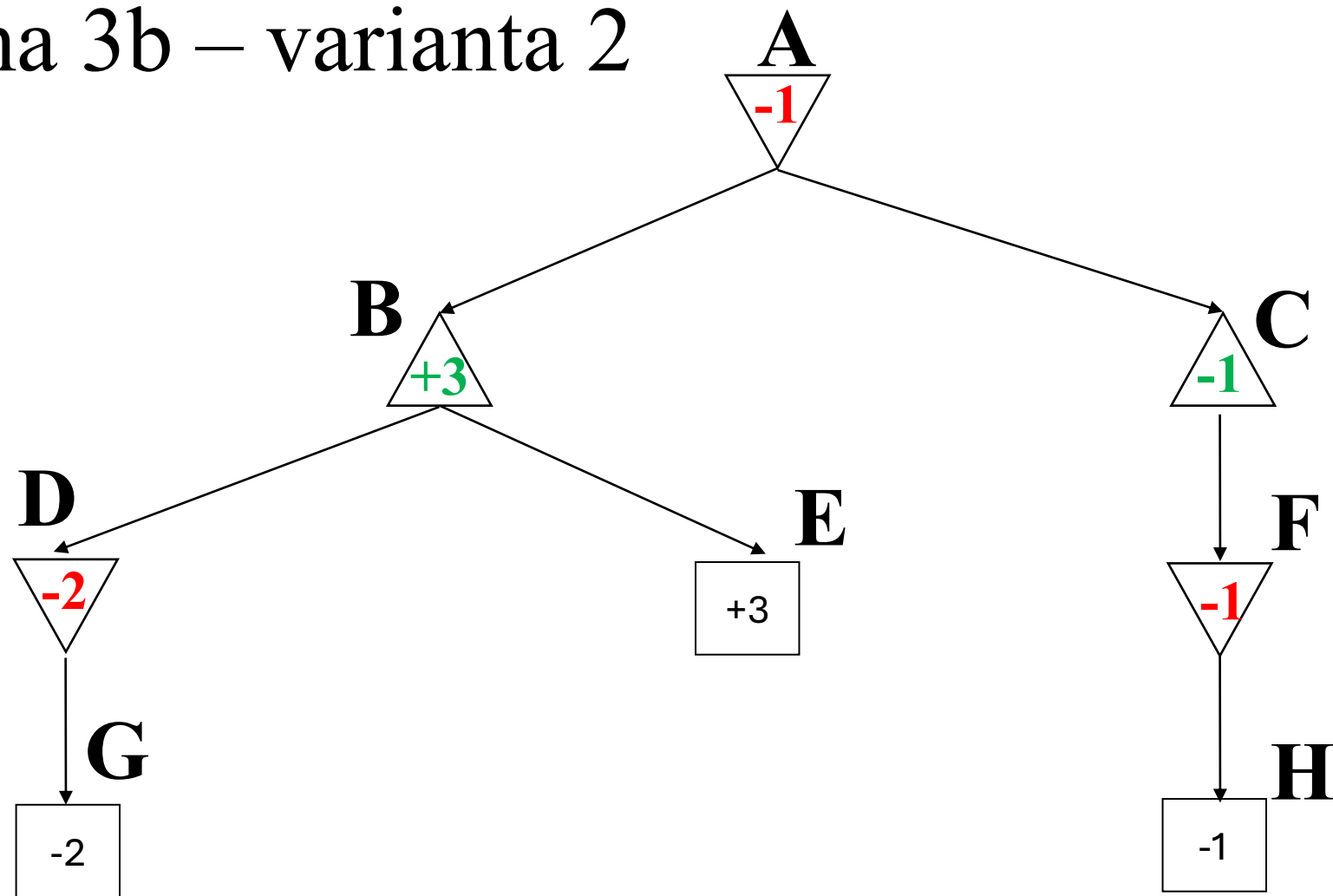
-2

Problema 3b – varianta 1

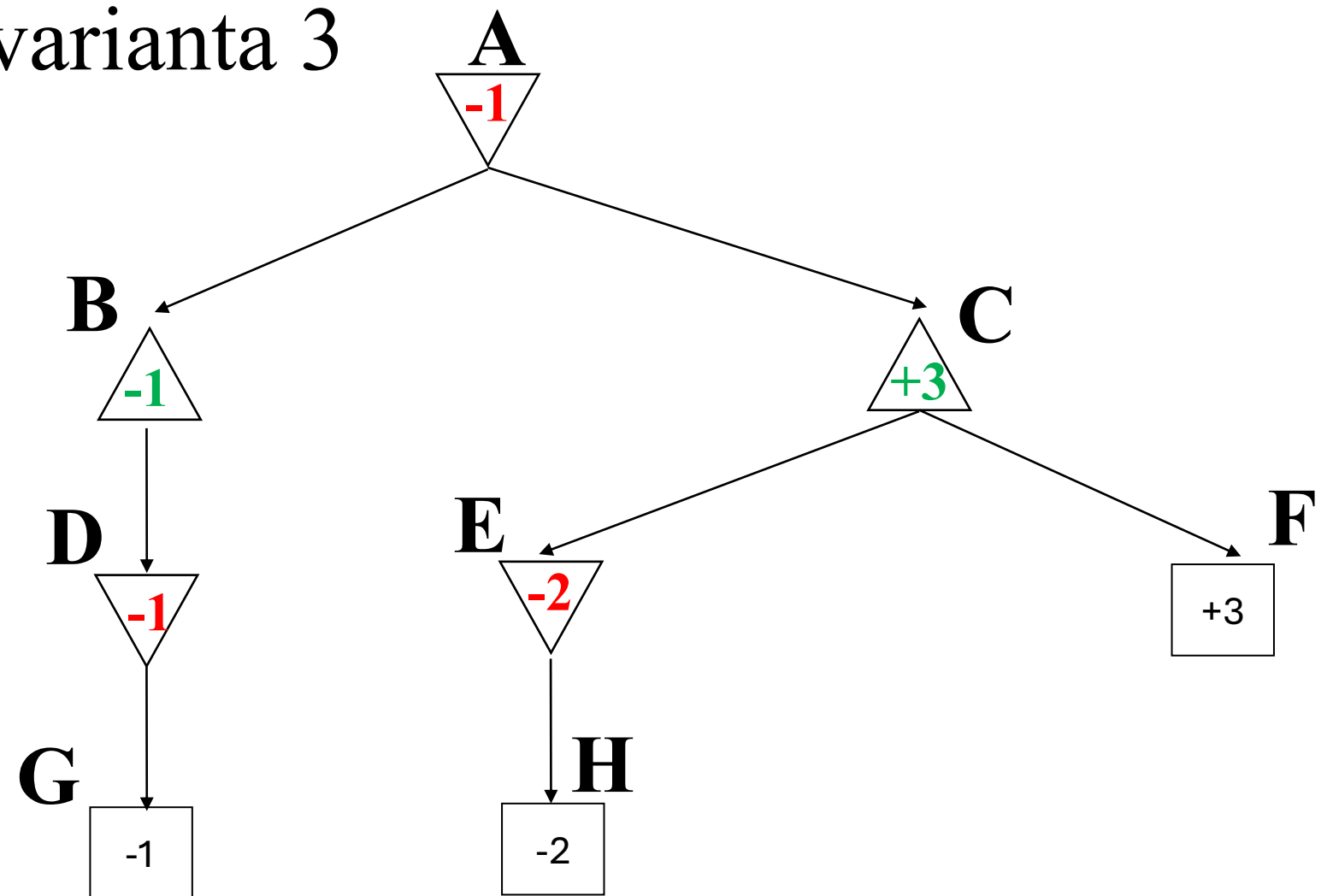


Pentru variantele 2, 3, 4 se obțin arbori similari

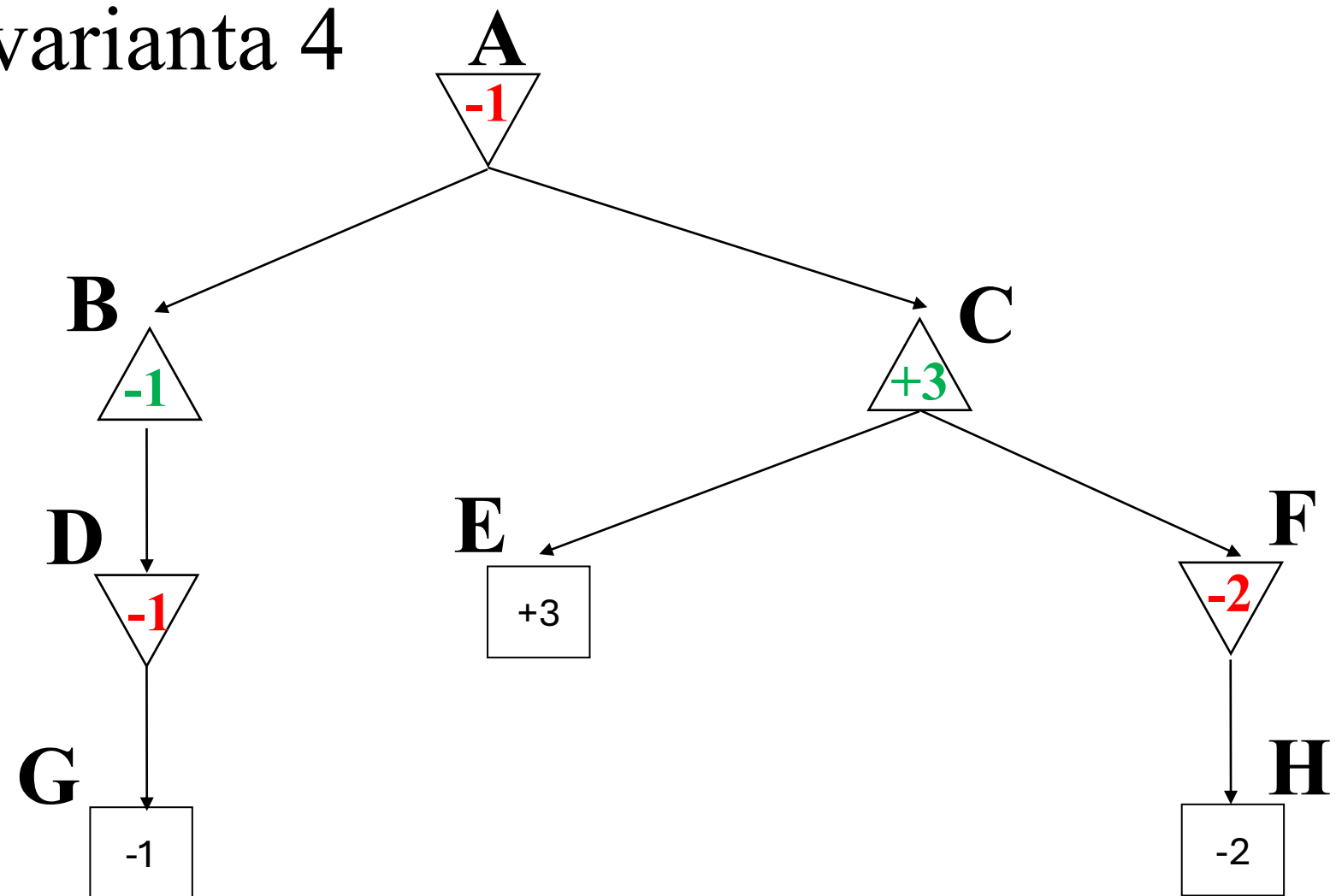
Problema 3b – varianta 2



Problema 3b – varianta 3



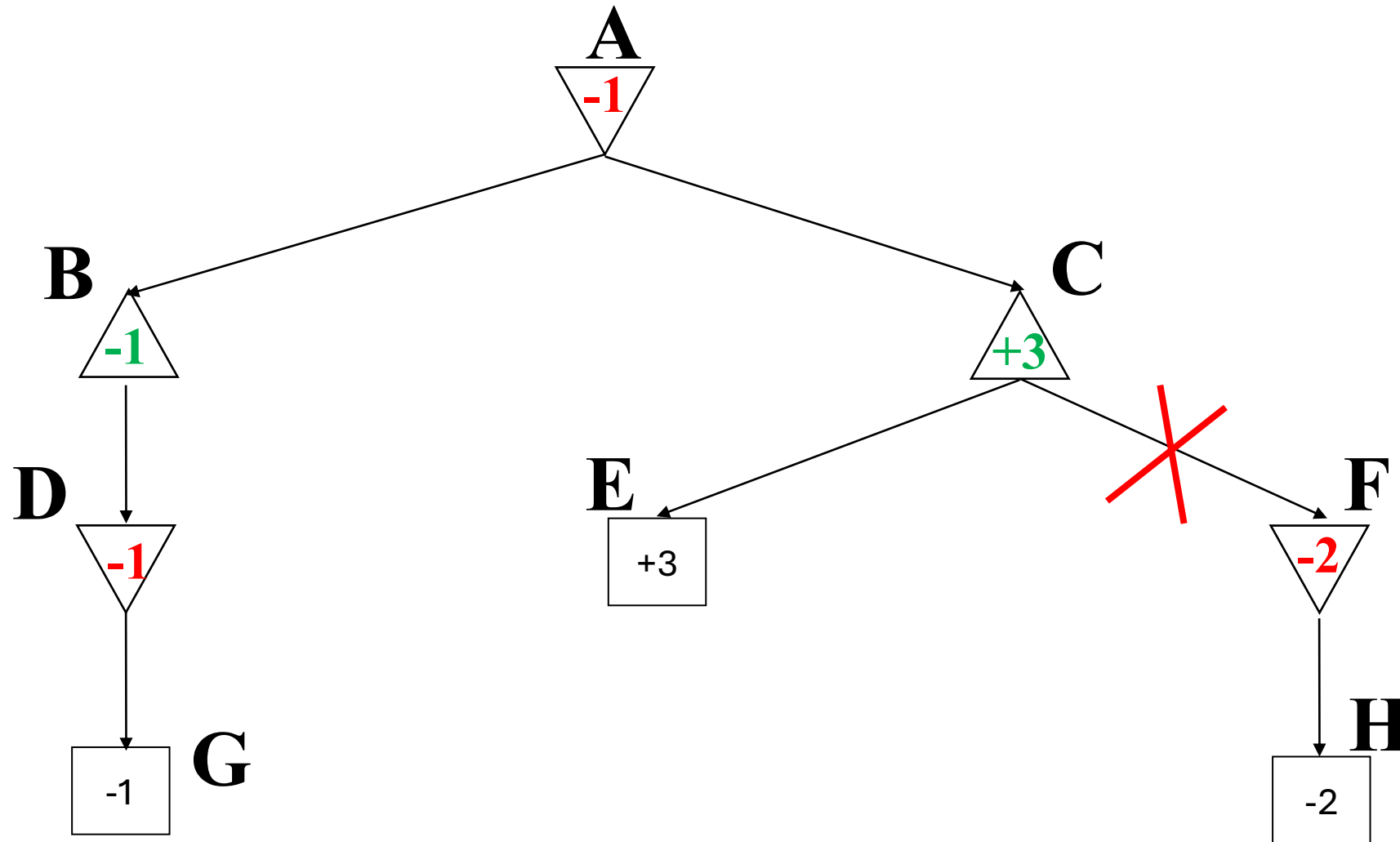
Problema 3b – varianta 4



Problema 3c

varianta 1, 2, 3

Pe arborele de la 3b nu putem face nicio retezare pentru alfa-beta. Putem reordona fiii, punând în subarborele stâng mutarea optimă, care conduce la minimul pentru MIN și maximul pentru MAX, vedeți arborele de mai jos. În acest caz nu mai explorăm nodurile F și H.



Problema 3c

varianta 4

Pe arborele de la 3b facem retezare pentru alfa-beta. În acest caz nu mai explorăm nodurile F și H. Nu mai putem reordona fiii, nu mai putem elimina alte noduri.

